

Docker – Introduction

Containerisation for Full Stack Development



DEVOPS & CLOUD



CONTAINERISATION

Why Do You Need Docker

Compatibility / Dependency

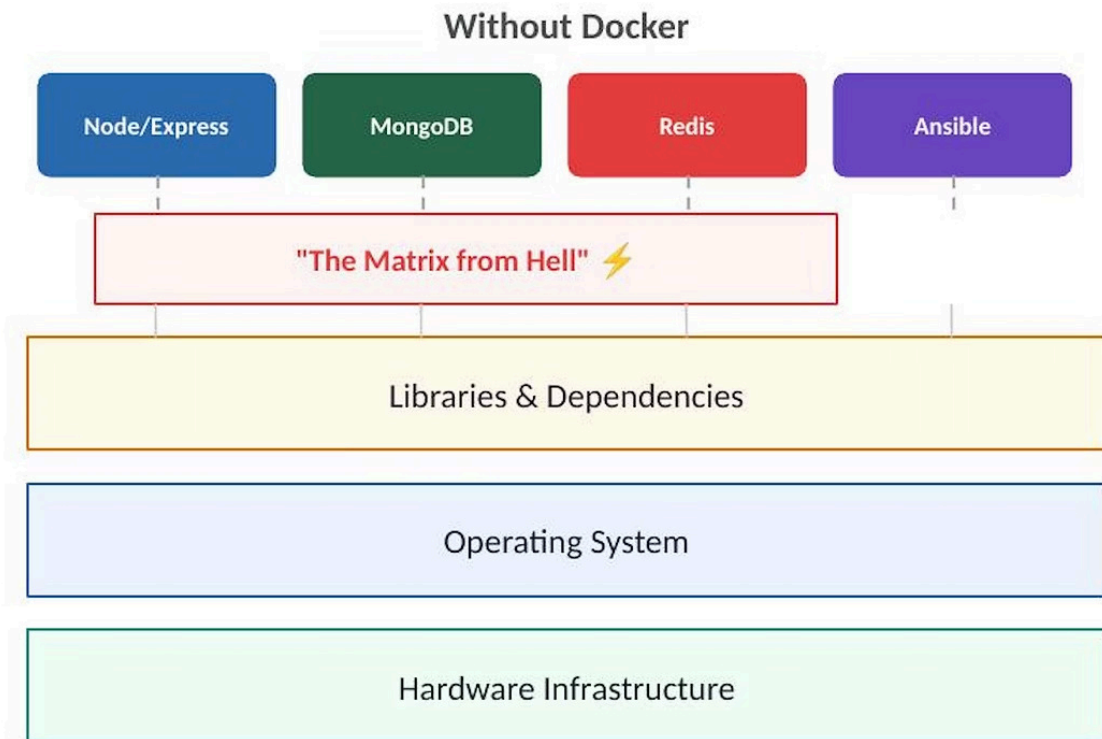
Different tools require different versions of libraries that may conflict with each other.

Long Setup Time

New team members spend days configuring environments before writing a line of code.

Dev / Test / Prod Drift

"Works on my machine" — inconsistencies across environments cause unpredictable bugs.



What Can Docker Do?

01 Containerize Applications

Package every service with its exact runtime, configs, and code.

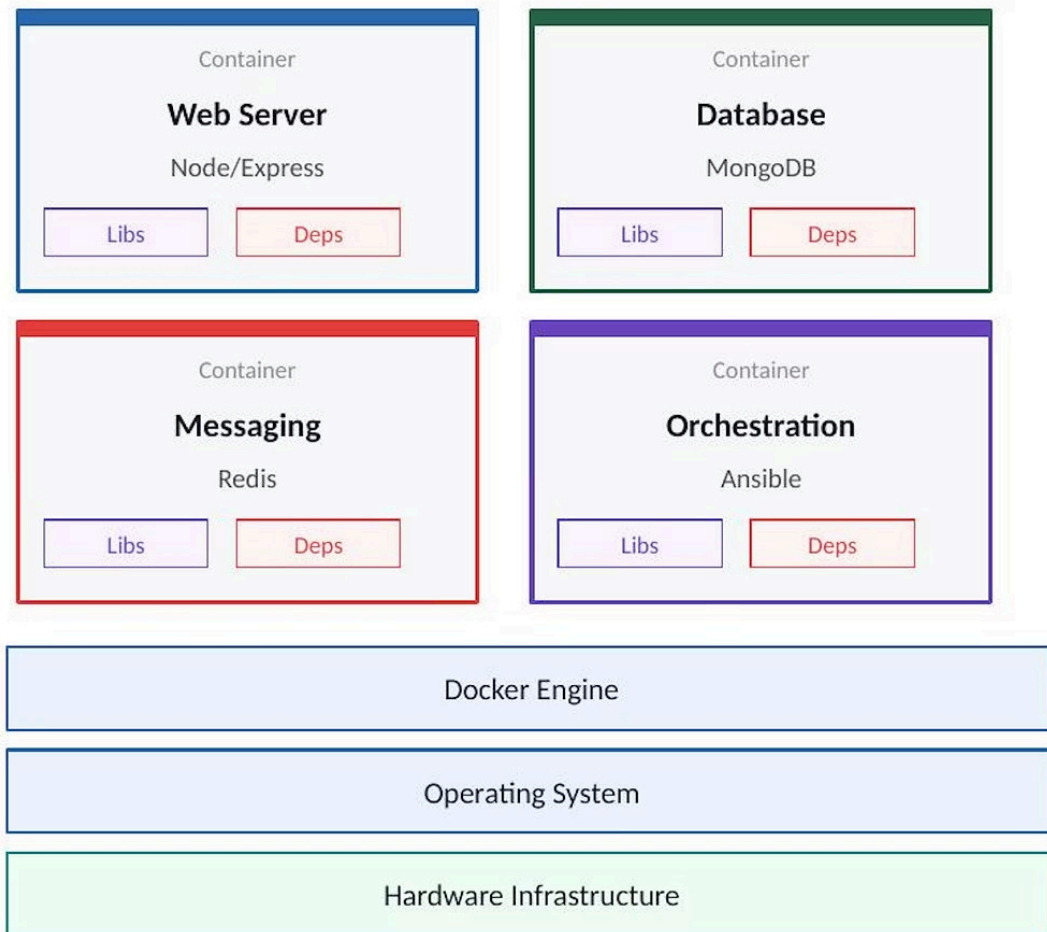
02 Isolated Dependencies

Each service carries its own libs and deps — zero conflicts.

03 Consistent Environments

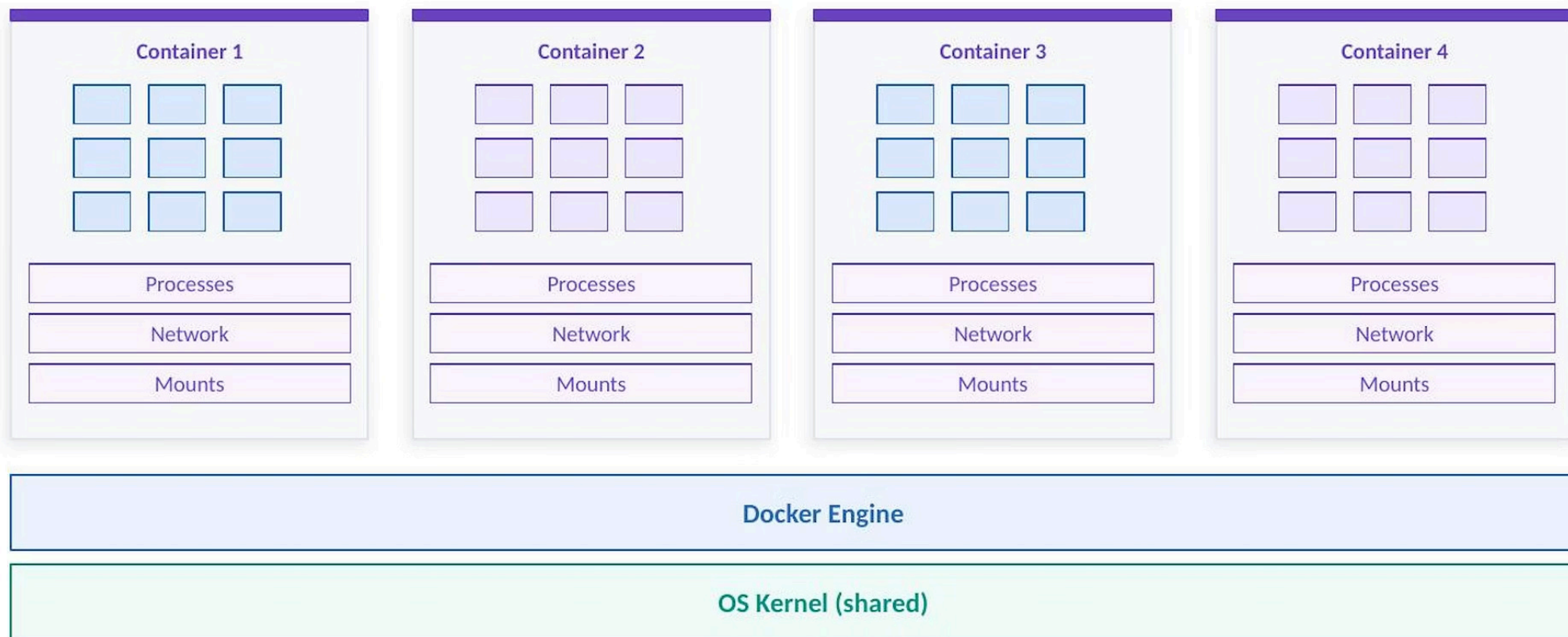
Same container runs identically on dev laptop, CI server, and production.

With Docker



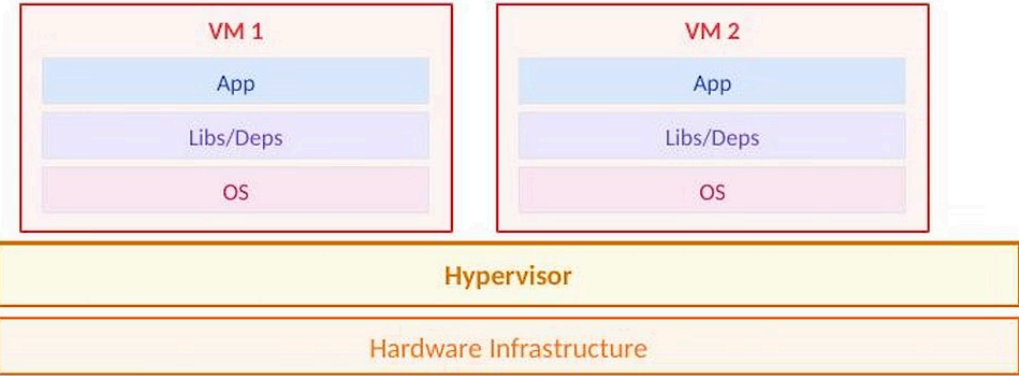
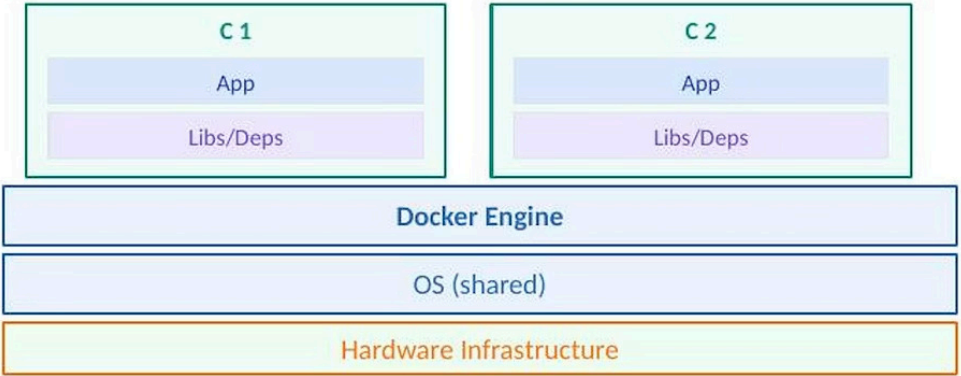
What Are Containers?

Containers isolate processes, network interfaces, and file mounts — sharing only the OS kernel.



Unlike VMs, containers share the host OS kernel — no full OS per container.

Containers V/s VMs

Virtual Machine		Docker Container	
Size	Gigabytes (GB)	✓	Megabytes (MB)
Boot Time	Minutes	✓	Seconds
Resource Usage	High — full OS per VM	✓	Low — shared kernel
Isolation	✓ Full OS isolation	Process-level isolation	
Architecture		Architecture	
 The diagram illustrates the Virtual Machine (VM) architecture. It shows two virtual machines, VM 1 and VM 2, each containing an App, Libs/Deps, and OS layer. These VMs are managed by a Hypervisor, which sits on top of the Hardware Infrastructure.		 The diagram illustrates the Docker Container architecture. It shows two containers, C 1 and C 2, each containing an App and Libs/Deps layer. These containers are managed by the Docker Engine, which sits on top of a shared OS layer, which in turn sits on top of the Hardware Infrastructure.	

THE PROBLEM

Why Do We Need Docker?

The Classic Pain Points

"It works on my machine!"

The most dreaded phrase in software development — environment mismatches cause endless frustration across teams.

- Environment mismatch between dev and production
- Painful dependency setup for every new team member
- Inconsistent deployments across servers

✓ Docker Solves This

Docker eliminates environment inconsistencies by packaging your application and all its dependencies into a single, portable unit that runs identically everywhere.

- ✓ No more "works on my machine" — Docker guarantees consistent behaviour across every environment, from a developer's laptop to a production cloud server.

FOUNDATION

What is Docker?



Open-Source Platform

Docker is a freely available, open-source platform maintained by a large global community of contributors and backed by Docker Inc.



Packages Everything

Bundles your application code together with all its runtime dependencies, libraries, and configuration into a single deployable image.



Uses Containerisation

Docker leverages containerisation technology to isolate applications and their dependencies into self-contained, portable units.



Runs Anywhere

Deploy on a laptop, on-premise server, or any major cloud provider — the container behaves identically in every environment.

CORE CONCEPT

What is a Container?

A container is a **lightweight, standalone, executable package** that includes everything needed to run a piece of software.

Code

Your application source code and compiled artefacts, exactly as written and tested by the development team.

Runtime

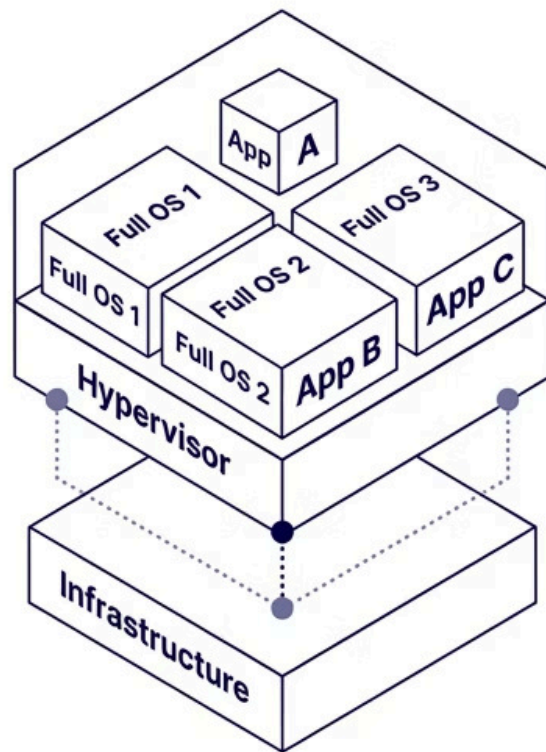
The execution environment — such as the JVM, Node.js runtime, or Python interpreter — required to run your application.

Libraries

All third-party libraries, frameworks, and system dependencies your application relies upon, pinned to exact versions.

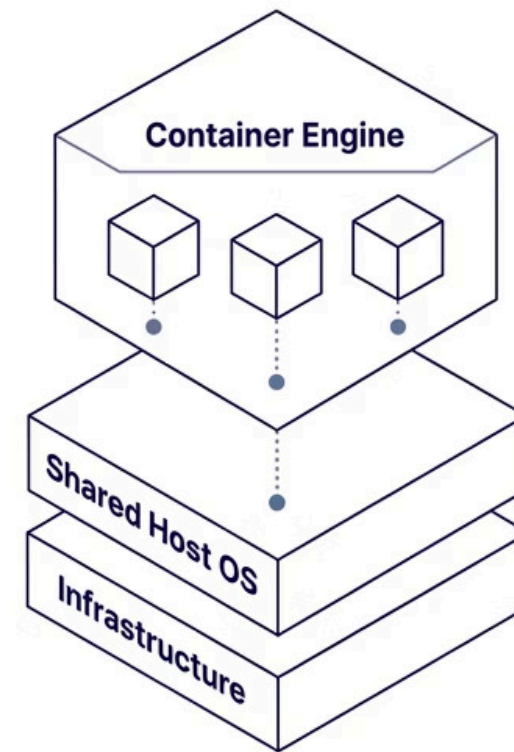
 Containers run **independently** and exhibit the **same behaviour across all environments** — development, staging, and production alike.

Containers vs Virtual Machines



Virtual Machines require a full OS per instance, are heavy in resources, and have slow startup.

Text is large and bold



Containers share the host OS, are lightweight with minimal overhead, and have millisecond startup.

Containers share the host operating system kernel, making them significantly more efficient than virtual machines which each require a full OS installation. This results in faster startup times, lower memory usage, and higher density on the same hardware.

BENEFITS

Key Benefits of Docker



Consistent Environment

Identical behaviour from development through to production — no more environment-specific bugs.



Faster Development

Spin up a fully configured development environment in seconds rather than hours of manual setup.



Easy Deployment

Ship your containerised application to any server or cloud platform with a single command.



Better Resource Utilisation

Containers share the host OS, consuming far fewer resources than traditional virtual machines.



Scalable Applications

Effortlessly scale individual services up or down to meet demand using container orchestration tools.

Docker Concepts



Image

The **blueprint** of your application — a read-only template containing all instructions to create a container.



Container

A **running instance** of an image — the live, executing version of your packaged application.



Dockerfile

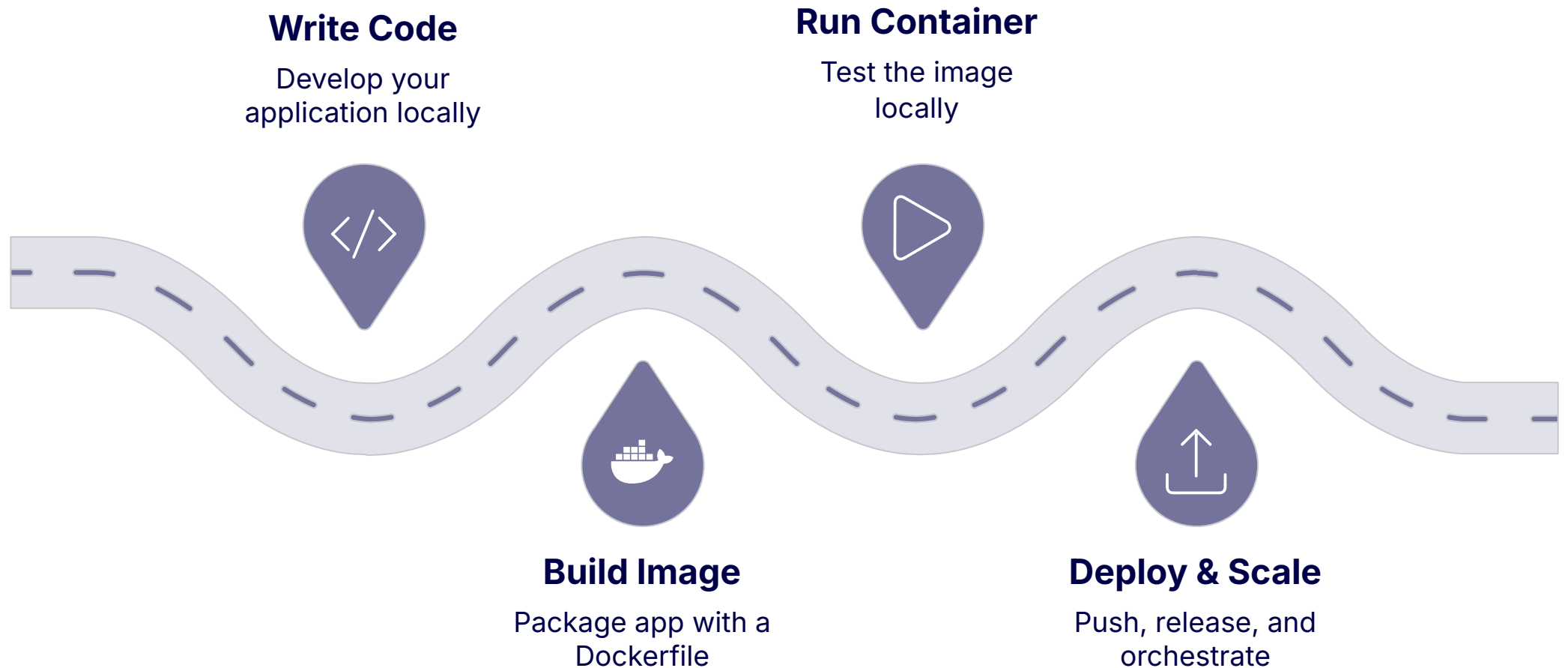
A text file containing **step-by-step instructions** that Docker uses to automatically build an image.



Docker Compose

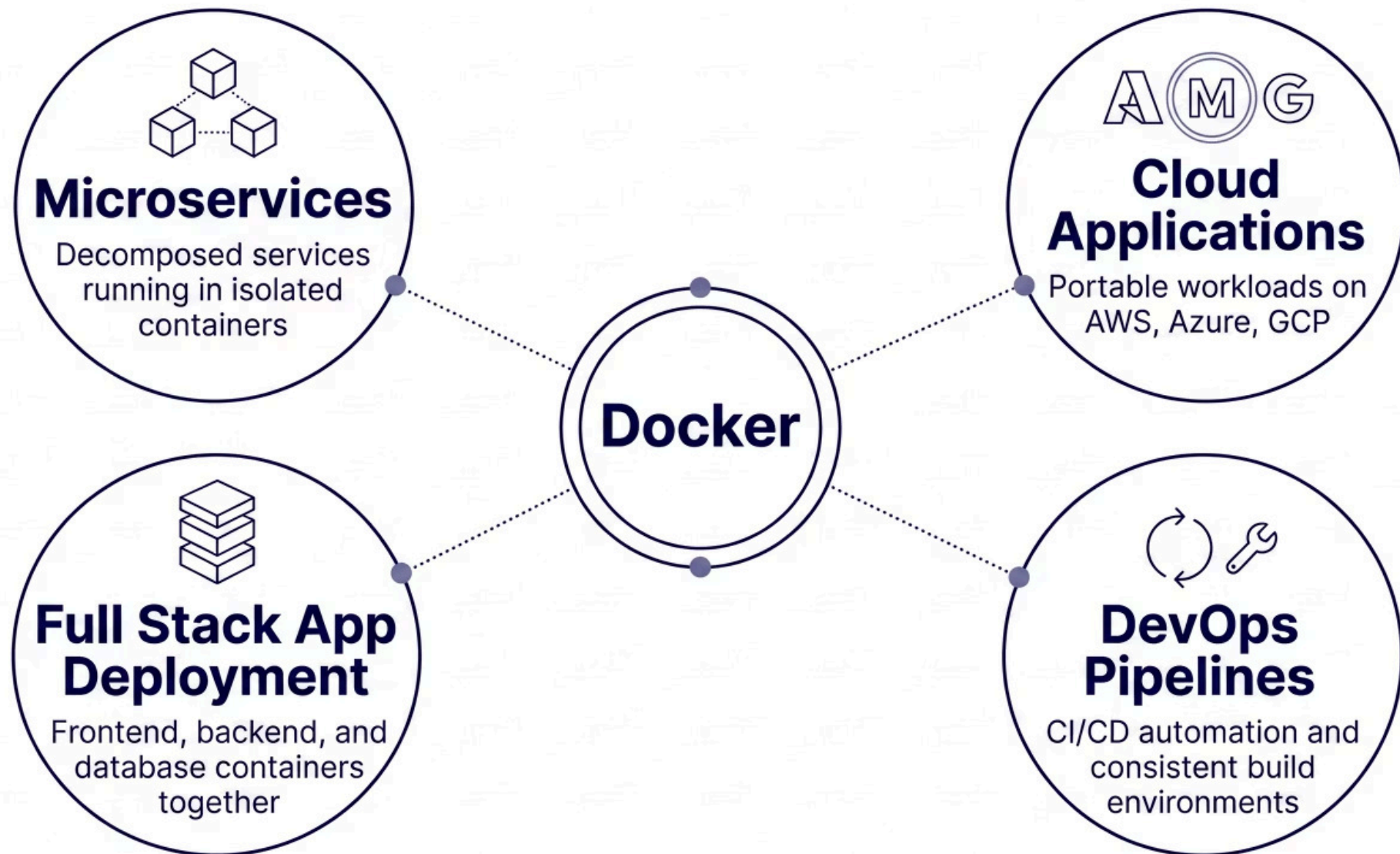
A tool for defining and running **multi-container** Docker applications using a single YAML configuration file.

Docker Workflow



The Docker workflow streamlines the entire software delivery lifecycle — from writing code on your local machine to deploying and scaling in production. Each step is repeatable, automated, and consistent regardless of the target environment.

Where Docker is Used



Docker has become the de facto standard across the modern software industry. Whether you are building microservices, deploying to the cloud, automating DevOps pipelines, or shipping full stack applications, containers are at the heart of the workflow.

REAL-WORLD EXAMPLE

Docker in Practice

✗ Without Docker

A team builds a web application. Each developer has a slightly different local setup — different OS versions, different library versions, different environment variables.

- Developer A's code works locally but fails on Developer B's machine
- Staging environment behaves differently from production
- Deployment errors are frequent and hard to diagnose
- Onboarding a new team member takes days of setup

✓ With Docker

The entire application — frontend, backend, and database — is containerised. Every team member runs the exact same environment with a single command.

- Identical behaviour on every developer's machine
- Staging and production environments are mirror images
- Deployments are predictable and repeatable
- New team members are productive within minutes

Docker in Your Training Journey

What You Will Learn

01

Docker Basics

Understand the core concepts of containerisation, how Docker works under the hood, and why it matters for modern development.

02

Images & Containers

Learn to pull images from Docker Hub, build your own custom images, and manage running containers effectively.

03

Docker Commands

Master the essential CLI commands for building, running, inspecting, stopping, and removing containers and images.

04

Docker Compose

Define and orchestrate multi-container applications — such as a Spring Boot backend with a PostgreSQL database — using a single YAML file.

Where It Fits

 Docker is a core component of the **DevOps / Cloud module** in your SPE Java Full Stack Engineer training programme.

The skills you build with Docker directly underpin your ability to deploy, manage, and scale the full stack applications you develop throughout the programme. Containerisation knowledge is a **critical requirement** for modern backend and cloud engineering roles.

SUMMARY

Summary

Docker = Containerisation Platform

Docker is an open-source platform that uses containerisation to package applications and their dependencies into portable, self-contained units.

Ensures Consistency

By packaging everything an application needs, Docker guarantees identical behaviour across development, staging, and production environments — eliminating the "works on my machine" problem for good.

Simplifies Deployment

Containerised applications can be deployed to any server or cloud platform with minimal effort, making releases faster, safer, and more predictable.

Critical for Modern Backend & DevOps

Docker is an industry-standard tool used across microservices, cloud platforms, and CI/CD pipelines — an essential skill for any full stack or DevOps engineer.

Hands-On: Get Started with Docker

Put theory into practice with these three essential first steps.



Install Docker

Download and install Docker Desktop for your operating system from **docker.com**. Verify the installation by running `docker --version` in your terminal.



Run Your First Container

Execute `docker run hello-world` to pull your first image from Docker Hub and run your first container. Explore basic commands: `docker ps`, `docker images`, `docker stop`.



Dockerise Your Spring Boot App

Write a `Dockerfile` for your existing Spring Boot application, build an image with `docker build`, and run it as a container — your app is now fully portable and production-ready.



Once you can Dockerise your Spring Boot app, you are ready for the full DevOps / Cloud module — including Docker Compose, CI/CD pipelines, and cloud deployment.